ÉPREUVE TECHNIQUE DE SÉLECTION

Plateforme bancaire distribuée Spring Cloud, IA, Kafka & Angular

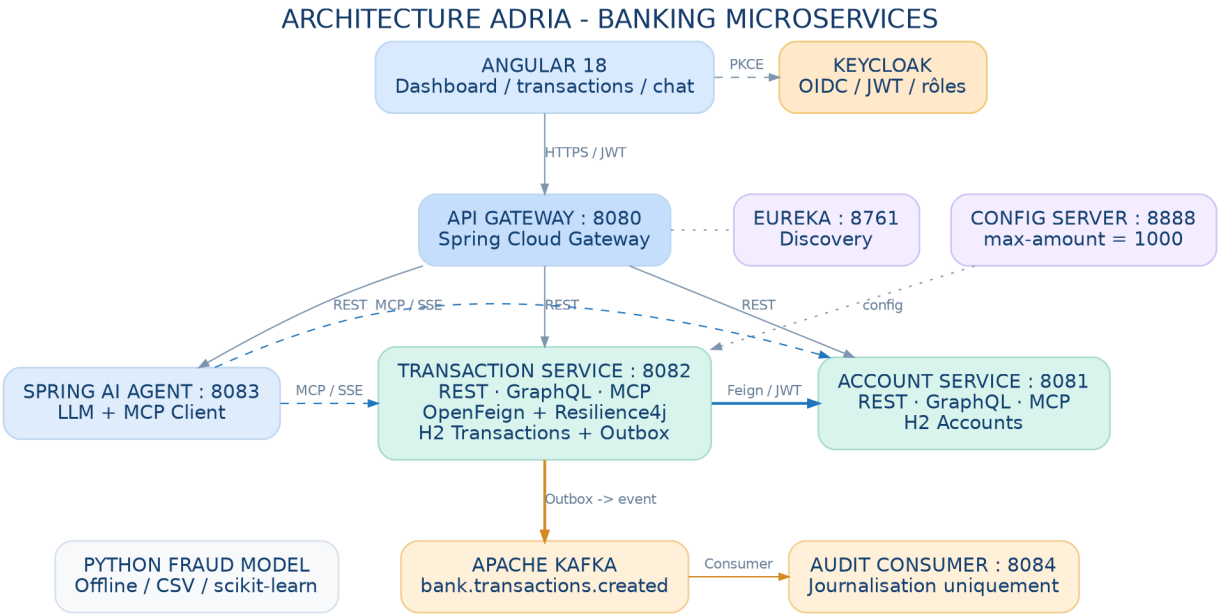
Priorité de candidature : Data Science (partie H) | Second choix : Full Stack

Rapport de conception, développement et résultats vérifiés de machine learning

Énoncé : Test.pdf (4 pages) — architecture micro-services, IA et développement full-stack, durée annoncée : 3 heures.

Référence pédagogique : Mohamed Youssfi, ENSET — architecture en couches, Spring Boot, Spring Cloud et pratiques Kafka.

Livrables : backend Maven multi-modules, frontend Angular (partie G), Python et notebook Jupyter exécuté (partie H prioritaire), Docker Compose, diagrammes, README et procédures de captures.



VALIDATION : Le fichier bank_transactions.csv fourni contient 5 000 lignes et 257 fraudes (5,14 %). Pipeline Python exécuté et évalué sur test indépendant, 6 tests pytest réussis. En revanche, aucune exécution Maven, Docker ou Angular n’est revendiquée sans preuve réelle.

1. Analyse du besoin et périmètre

L’application bancaire a pour fonctions la consultation des comptes, l’enregistrement de transactions et les virements. Chaque transaction respecte le statut et la devise du compte, le solde disponible et le plafond bancaire partagé. Les événements de transaction doivent être publiés dans Kafka ; un assistant agentique utilise les outils exposés via MCP. Un modèle de fraude est étudié séparément en Python.

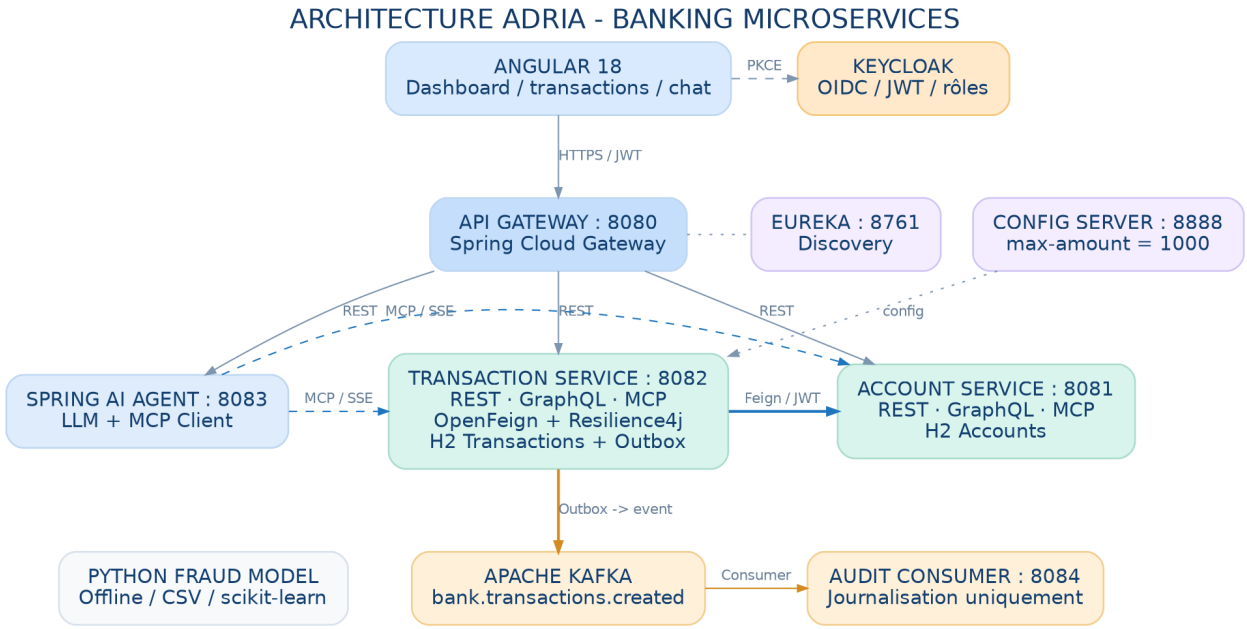
Correspondance avec les parties A à I de l’énoncé

Partie	Objectif	Implémentation / preuve
A	Remise, GitHub et rapport	README/PDF ; dépôt, Sheet, Classroom et commits à effectuer personnellement
B	Architecture et UML	Schémas PNG et sources DOT/SVG
C	Métier REST/GraphQL/MCP	Account, Transaction, OpenFeign, Resilience4j
D	Infrastructure Spring Cloud	Gateway, Eureka, Config Server ; max-amount = 1000
E	Agent conversationnel	Spring AI + client MCP, endpoint REST /api/chat
F	Événements Kafka	Outbox, topic, consumer de journalisation
G	Client web au choix	Dashboard Angular uniquement, sans Flutter
H (priorité 1)	Détection des fraudes	CSV fourni traité, 3 modèles en CV, résultats test, notebook exécuté
I	DevOps	Dockerfile, Compose, CI GitHub Actions, discussion de rollback

Choix de réalisation

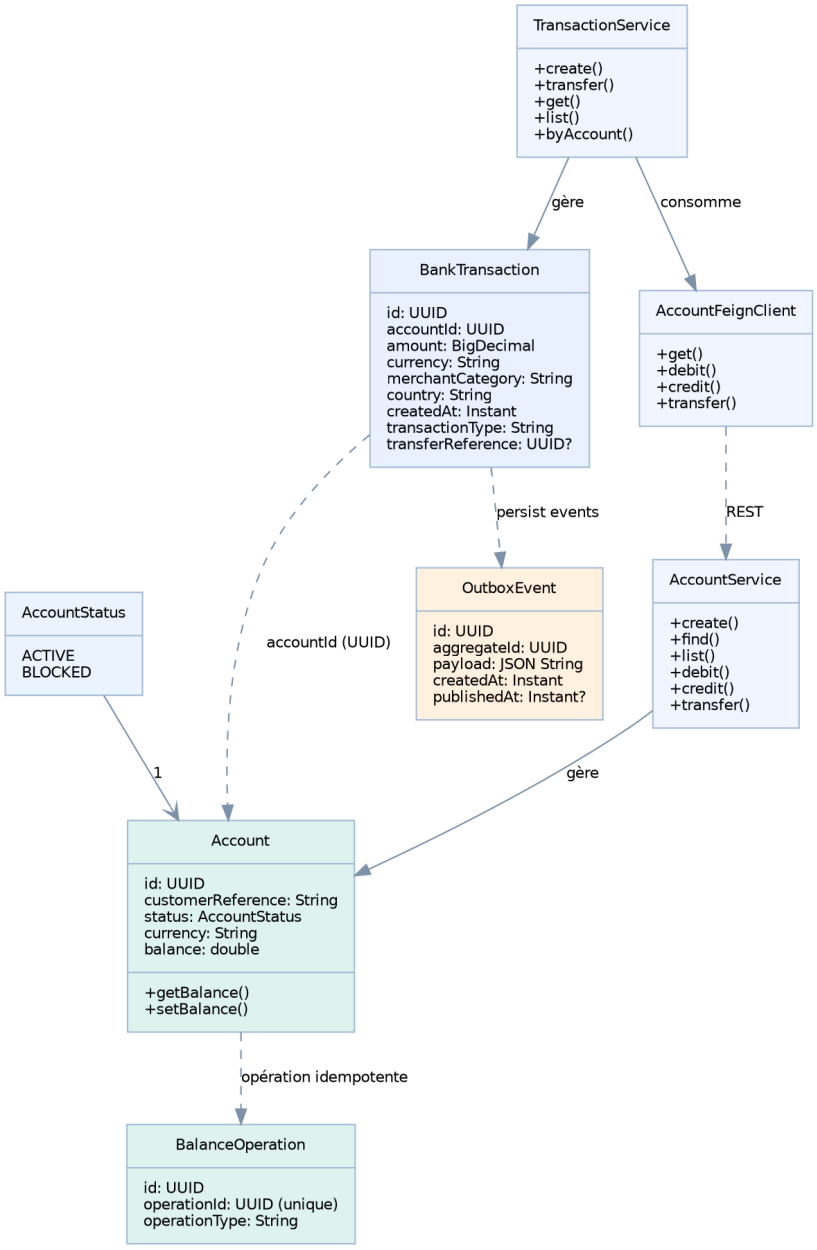
Deux microservices fonctionnels : AccountService et TransactionService. Les composants Discovery, Config, Gateway, AI Agent et Audit Consumer sont techniques. L’application Angular et Keycloak sont des services d’intégration ; le modèle Python est volontairement hors du runtime Spring, conformément à la consigne.

2. Architecture logique et UML



Les clients passent par Gateway (8080), qui utilise Eureka (8761). Le Config Server (8888) centralise le plafond des transactions. TransactionService (8082) interroge AccountService (8081) à travers OpenFeign et un circuit breaker Resilience4j. AI Agent (8083) utilise les deux outils MCP. Kafka reçoit les événements du TransactionService et l'Audit Consumer (8084) les journalise. Le modèle Python de fraude n'est pas connecté au broker.

DIAGRAMME DE CLASSES - DOMAINE BANCAIRE



Les classes BankTransaction et Account vivent dans deux bases séparées, reliées logiquement par accountId : UUID, pas par une jointure JPA. Cette isolation reprend le principe de responsabilité autonome de chaque microservice enseigné dans le cours.

3. Implémentation et séparation des couches

Couche	Fichiers / rôle
domain/	Account, AccountStatus, BalanceOperation, BankTransaction, OutboxEvent
repository/	Spring Data JPA : AccountRepository, TransactionRepository, OutboxRepository
service/	AccountService, TransactionService, DTO et implémentations ; règles métier
web/rest/	API HTTP : CRUD comptes, transactions, virements
web/graphql/	QueryMapping et MutationMapping ; délègue à la même couche Service
mcp/	@Tool + ToolCallbackProvider ; délègue à la même couche Service
client/	OpenFeign, relais Bearer, Resilience4j pour accès distant

Versions retenues

Java 21 ; Spring Boot 3.4.2 ; Spring Cloud 2024.0.0 ; Spring AI 1.0.3 ; Springdoc 2.8.4 ; Angular 18. Ces versions suivent l'approche du professeur et la compatibilité nécessaire aux fonctionnalités AI/MCP. Dépendances Maven réparties dans les modules et versions gérées par les BOM du parent.

Contrats et endpoints métier

Service	Routes / fonctionnalités
Account	GET/POST /api/accounts ; GET/PATCH /api/accounts/{id} ; appels internes débit/crédit/virement
Transaction	GET/POST /api/transactions ; GET /api/transactions/{id} ; POST /api/transfers
GraphQL	POST /graphql/accounts et /graphql/transactions via Gateway, requêtes et mutations
Agent	POST /api/chat ; message et booléen confirmWrites
Swagger	/swagger-ui/index.html sur ports 8081 et 8082

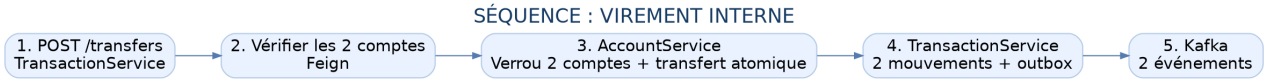
Vérification des règles métier

Les validations imposent : montant strictement positif, maximum 1000, compte actif, devise égale, solde suffisant, deux comptes distincts. Les comptes sont verrouillés lors du débit/virement. Les identifiants d'opération sont uniques pour empêcher la répétition de mouvements sur un même operationId.

4. Séquence de transaction et virement



Après validation, le solde est débité dans AccountService ; TransactionService persiste localement la transaction et son événement Outbox dans la même transaction JPA. Une tâche planifiée publie ensuite l'événement dans Kafka. En cas de panne du broker, l'Outbox conserve l'événement à réessayer.



Le virement entre deux comptes est atomique dans la base de AccountService, après verrouillage de chaque compte. Les mouvements de transfert sont ensuite ajoutés à TransactionService. Une compensation best-effort tente le mouvement inverse si cet enregistrement échoue ; elle ne garantit pas une ACID globale entre services.

Communication et résilience

OpenFeign identifie AccountService par son nom Eureka ; Resilience4j assure l'ouverture temporaire d'un circuit si le service distant est défaillant. Il ne faut jamais remplacer un solde réel indisponible par une valeur fictive de fallback. Les erreurs 4xx métier sont préservées lorsqu'elles proviennent du service distant.

5. Intégration REST, GraphQL, MCP, IA et Keycloak

Trois accès, une seule logique métier

REST, GraphQL et MCP utilisent les mêmes services applicatifs. REST est documenté par Springdoc ; les schémas GraphQL se trouvent dans src/main/resources/graphql ; Spring AI expose des outils @Tool via les serveurs MCP des modules métier. Le client MCP SSE de l'agent découvre les outils et les fournit au ChatClient.

Agent bancaire

POST /api/chat reçoit un message et un booléen confirmWrites. Les outils d'écriture sont exclus si confirmWrites vaut false. Un modèle LLM est sollicité uniquement avec une clé fournie à travers OPENAI_API_KEY. Le chatbot doit utiliser les outils pour connaître les vrais soldes et ne pas inventer les résultats. Le test REST est le client choisi ; Telegram et Discord n'ont pas été développés.

Sécurité et autorisations

Keycloak importe le realm bank et le client public bank-ui, qui emploie Authorization Code + PKCE. Les comptes de démonstration ont des rôles VIEWER et OPERATOR. Gateway et services métier disposent de la validation JWT. Les appels Feign relaient le Bearer token lorsqu'il existe ; le fallback service-to-service prévoit un client credentials. Les endpoints MCP SSE nécessitent un filtrage réseau rigoureux : ils ne doivent pas être publiés sans protections supplémentaires.

Tests attendus / captures

Insérer des captures réelles de Swagger (S05-S06), GraphQL (S09), connexion Keycloak (S13), une conversation agentique (S10), et Angular (S12). Le présent rapport ne fabrique pas ces captures.

6. Kafka et communication événementielle

Topic : bank.transactions.created. La clé Kafka est accountId. Un événement TransactionCreated comprend eventId, eventType, transactionId, accountId, amount, currency et occurredAt. Une tâche Outbox réessaie les publications après panne de Kafka. Le consumer écrit une ligne de log de l’événement reçu et n’accède à aucune base de comptes ou transactions.

Champ	Type / exemple
eventId	UUID aléatoire
eventType	TransactionCreated
transactionId	UUID de la transaction persistée
accountId	UUID utilisé aussi comme clé Kafka
amount	250.00 (nombre)
currency	EUR
occurredAt	Instant UTC ISO-8601

Limites et garanties

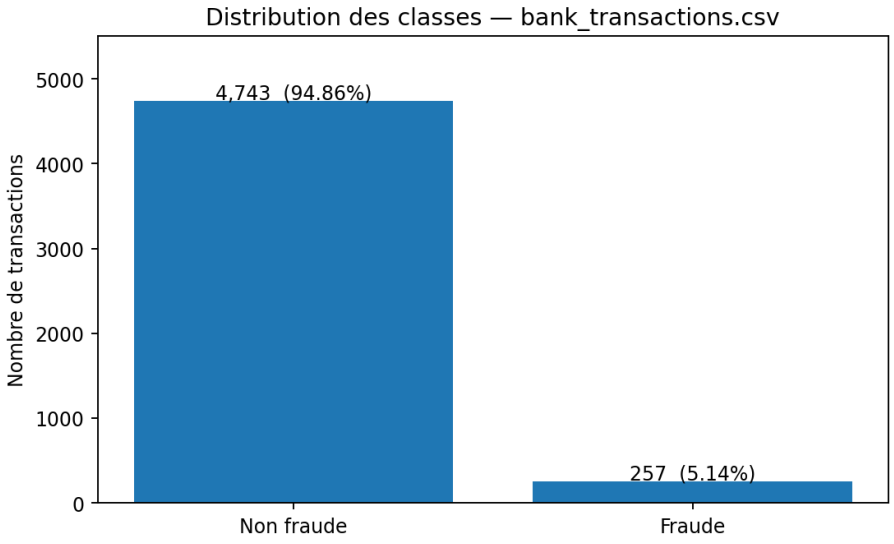
L’Outbox évite la perte d’événement entre persistance JPA et Kafka, mais la livraison est de type at-least-once. Il faut dédupliquer les eventId côté consommateur en production ; les logs actuels sont uniquement des journaux pédagogiques. Référence de capture : S11 via docker compose logs event-consumer-service.

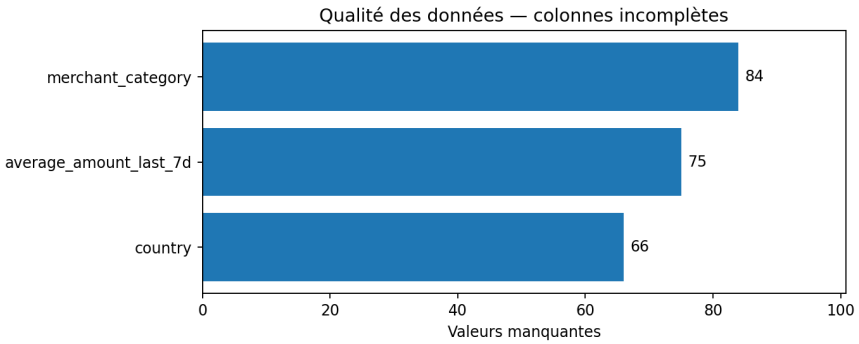
7. PARTIE H — Data Science : données, méthode et résultats

Premier choix de stage : Data Science. Cette section rend compte de résultats effectivement calculés à partir du fichier bank_transactions.csv fourni (5 000 lignes, 9 variables au total, 257 cas de fraude). Le modèle reste expérimental, hors microservices, conformément à l'énoncé. Les parties techniques conservent leur architecture Spring Cloud et le frontend G est uniquement Angular.

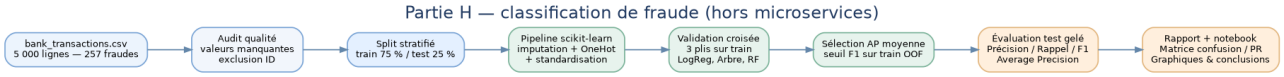
7.1 Profil des données : qualité et déséquilibre

Indicateur	Valeur observée
Transactions	5 000
Non fraudes	4 743 = 94,86 %
Fraudes	257 = 5,14 %
Attributs	9 dont is_fraud et deux identifiants
Valeurs manquantes	country : 66 ; merchant_category : 84 ; average_amount_last_7d : 75
IDs en double	0 identifiants transaction répétés





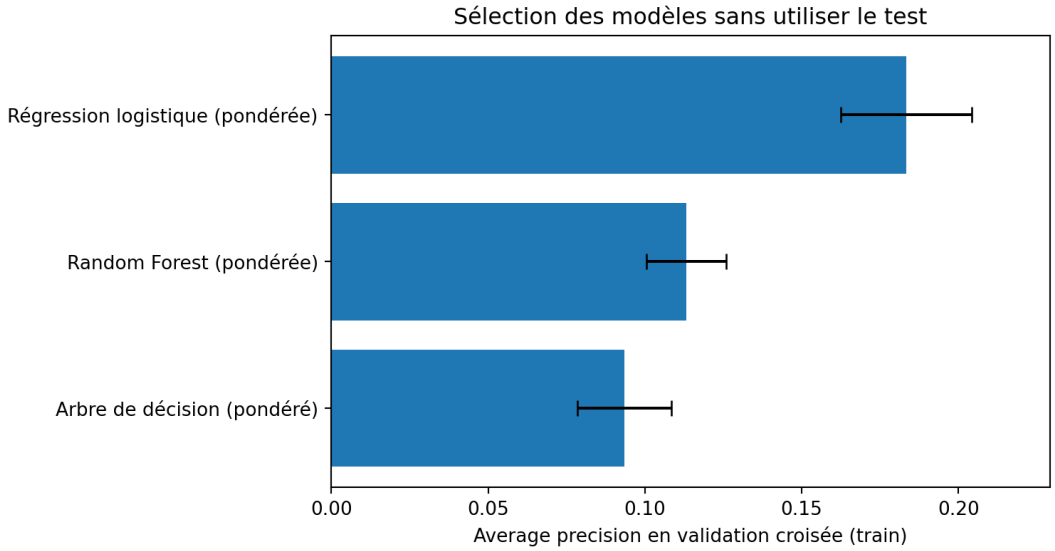
7.2 Méthode anti-fuite et sélection en validation croisée



Le découpage stratifié, seed 42, réserve 3 750 transactions à l'entraînement et 1 250 au test (64 fraudes). Une chaîne scikit-learn impute les numériques par médiane et les catégories par mode, applique la standardisation et le OneHotEncoder. Chaque prétraitement est appris dans les seuls plis d'entraînement. Les identifiants transaction_id/account_id et la cible sont exclus des variables explicatives.

Comparaison des modèles : average precision en CV stratifiée 3 plis sur train

Modèle	AP moyenne train	Écart-type
Régression logistique pondérée	0,1833	0,0209
Random Forest pondérée	0,1131	0,0127
Arbre de décision pondéré	0,0935	0,0150

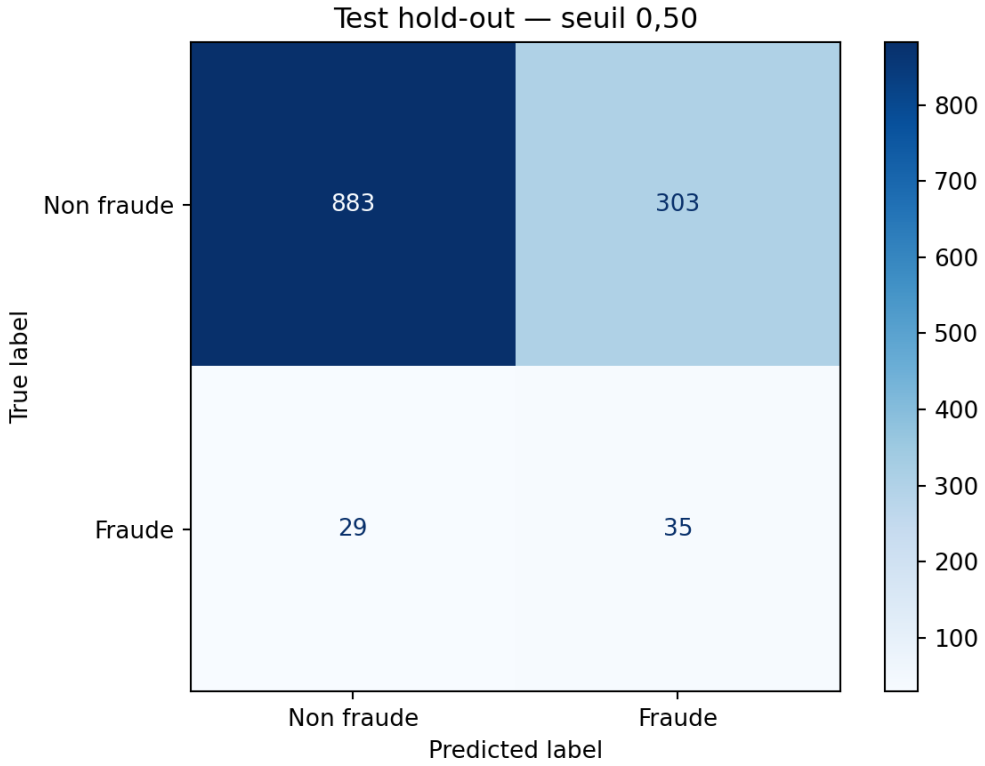


La régression logistique pondérée est sélectionnée sur le train uniquement. Le seuil fixe 0,50 est la référence ; le seuil secondaire 0,80 provient de la maximisation du F1 sur prédictions hors-pli du train. Aucun choix n'utilise les étiquettes du test.

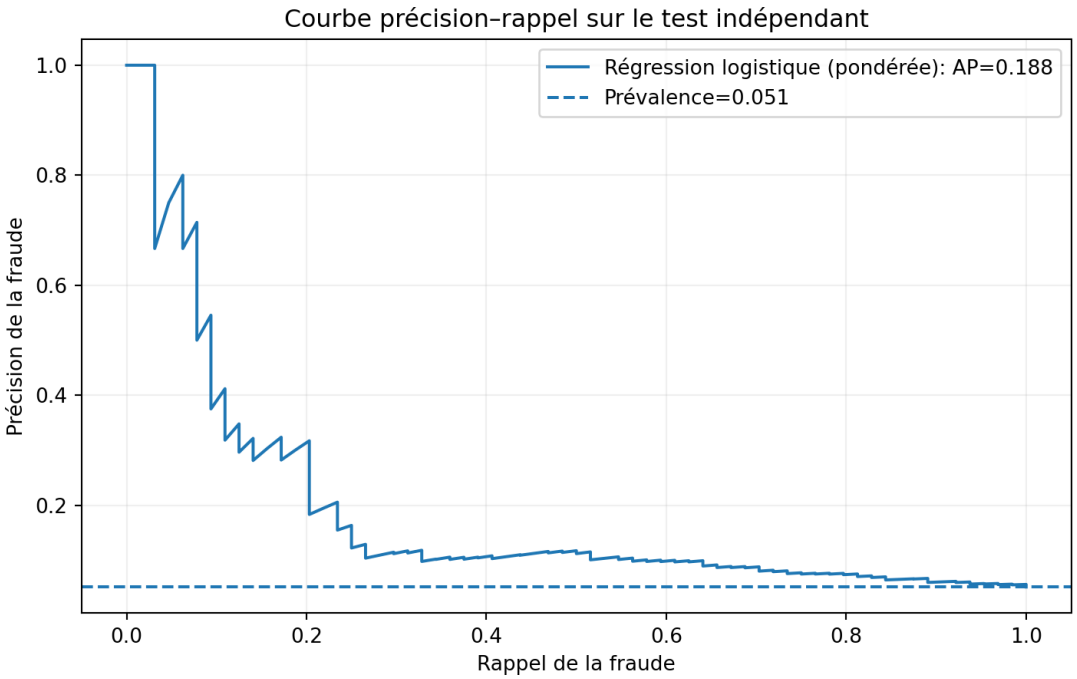
7.3 Évaluation indépendante sur les 1 250 transactions de test

Approche	Accuracy	Précision fraude	Rappel fraude	F1 fraude
Baseline non fraude	94,88 %	0 %	0 %	0
Modèle seuil 0,50	73,44 %	10,36 %	54,69 %	0,1741
Modèle seuil 0,80	92,48 %	23,21 %	20,31 %	0,2167

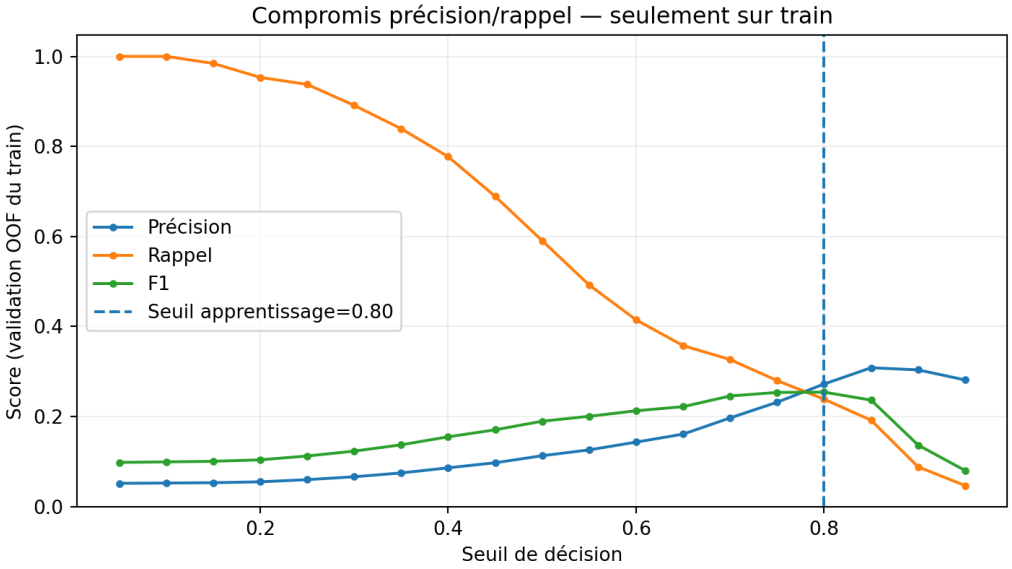
Average Precision test : 0,1885. Pour le seuil de 0,50 : 883 vrais négatifs, 303 faux positifs, 29 faux négatifs et 35 vrais positifs. Le modèle détecte 35/64 fraudes, mais produit 303 fausses alertes. Il ne peut pas être présenté comme prêt à déployer.



7.4 Courbe précision-rappel, seuil et interprétabilité

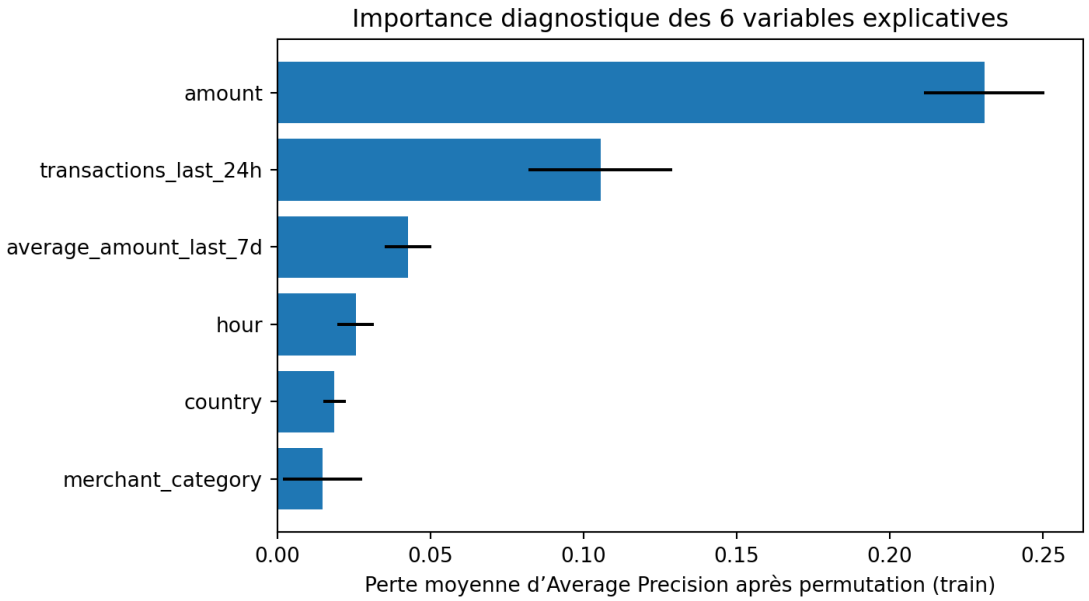


La performance doit être comparée à la prévalence de référence (environ 5,14 %). Average Precision = 0,1885 sur le test. Une courbe PR est plus informative qu'une accuracy globale dans le cas de fraude minoritaire.



L'étude du seuil est calculée uniquement avec les probabilités hors-pli de l'entraînement. Le seuil choisi sur train est 0,80 ; sur le test il conduit à 43 fausses alertes contre 303 à 0,50, mais augmente les fraudes manquées de 29 à 51. Le compromis doit être défini selon les coûts métier.

7.5 Importance des variables et questions d’analyse



L’importance par permutation est calculée sur le train, à des fins diagnostiques. Elle ne démontre pas de relation causale entre les variables et les fraudes.

Réponses aux quatre questions de l’épreuve

1. Accuracy insuffisante : la stratégie toujours non-fraude donne 94,88 % d’accuracy mais ne détecte aucune des 64 fraudes du test. Considérer rappel, précision, F1 et AP.

2. Faux positifs et fraudes manquées : seuil 0,50 = 303 fausses alertes et 29 fraudes manquées. Seuil 0,80 = 43 fausses alertes et 51 fraudes manquées. Aucun arbitrage sans coûts métier.

3. Diminution du seuil : de 0,80 à 0,50, le rappel monte de 20,31 % à 54,69 %, mais les faux positifs augmentent de 43 à 303 sur le test.

4. Validation temporelle : entraînement sur le passé et test sur le futur réduisent les fuites et mesurent mieux le déploiement. Le CSV n’a pas de timestamp complet, seulement hour ; la validation temporelle est donc impossible avec ces données. La stratification demandée est appliquée.

Livrables Data Science exécutés

fraud/train.py ; fraud/Analyse_Fraude_ADRIA.ipynb (notebook avec sorties réelles) ; fraud/results/metrics.json, profile et figures ; 6 tests unitaires Python réussis. Ni service REST de modèle, ni intégration Kafka : non demandés par la partie H.

8. DevOps et déploiement local

Docker Compose

Le fichier compose.yml rassemble ZooKeeper, Kafka, Keycloak, Eureka, Config, Gateway, Account, Transaction, Audit Consumer, AI Agent et Angular. Un Dockerfile Maven multi-stage prend le paramètre MODULE et utilise Java 21. Les ports publics de développement sont associés à 127.0.0.1. Le démarrage de conteneurs ne prouve pas leur disponibilité : vérifier les health endpoints et les logs.

Pipeline CI/CD — GitHub Actions

La CI exécute la compilation Maven, les tests unitaires, le build Angular, la construction des images et leur publication sur GitHub Container Registry avec un tag SHA. La phase de déploiement reste un exemple désactivé, car aucun déploiement cloud réel n’est exigé. Les variables de production doivent être créées dans GitHub Secrets / un gestionnaire de secrets.

Exigence DevOps	Réponse du projet
Secrets et configuration	Secrets via variables sécurisées / GitHub Secrets ; configuration partagée dans Spring Config, sans clé réelle dans Git.
Démarrage, disponibilité, vitalité	Actuator /actuator/health ; probes readiness/liveness à finaliser pour le runtime cible.
Logs, métriques, traces	Logs SLF4J + Actuator ; Prometheus/Grafana/OpenTelemetry recommandés pour la production.
Rollback	Images immuables SHA ; redéployer le digest de la dernière version saine.

9. Plan de tests, preuves et captures réelles

Les tests Java nécessitent Maven et les dépendances réseau ; les tests de conteneurs nécessitent Docker et Keycloak. Ces exécutables ne sont pas disponibles ici. Le tableau indique les preuves à produire par l’étudiant après exécution locale, sans validation non observée.

Réf.	Capture / test attendu
S01 / S02	Arborescence Maven et résultat réel mvn clean verify
S03 / S04	Eureka et configuration transaction-service (1000)
S05 / S06	Swagger Account et Transaction
S07 / S08	Transaction valide, refus montant 1001, fonds insuffisants
S09 / S10	GraphQL, puis agent MCP avec clé LLM valide
S11 / S12	Log consumer Kafka et dashboard Angular
S13 / S14	Connexion Keycloak et docker compose ps
S15 / S16	Résultats Python réels fournis ; GitHub Actions à exécuter sur votre dépôt

Protocole d’essai

Commencer par une transaction nominale de 250 EUR sur un compte actif disposant d’un solde supérieur. Vérifier le nouveau solde, l’enregistrement dans TransactionService, puis l’événement consommé. Refaire avec 1001 EUR, une devise différente et un compte bloqué. Vérifier ensuite les virements, GraphQL, Keycloak et le chatbot. Le README détaille les URLs et commandes.

10. Conclusion, améliorations et limites

Le projet propose un démonstrateur cohérent de système bancaire distribué : séparation des responsabilités, service Discovery, configuration centralisée, Gateway, règles métier, OpenFeign/Resilience4j, GraphQL/MCP, Spring AI, Kafka avec Outbox, sécurité Keycloak, Angular, ML Python et livrables CI/CD. Il suit l'approche en couches du cours et répond aux parties A à I du sujet par du code, des schémas et des procédures.

Limites à préciser au jury

• Tests Java, build Angular et conteneurs non exécutés ici : effectuer ces vérifications sur votre poste. Tests Python sur scénarios artificiels : 6/6 réussis.

• Le CSV fourni a été évalué : AP test 0,1885 et rappel 54,69 % à seuil 0,50, mais 303 faux positifs. Modèle volontairement offline, sans déploiement.

• Solde stocké en double selon le modèle minimal ; recommander BigDecimal/DECIMAL en production.

• Pas de transaction distribuée ACID : compensation best-effort et besoin de Saga/réconciliation.

• Outbox et Kafka entraînent la possibilité de messages doublés ; déduplication par eventId à ajouter.

• MCP SSE, Keycloak et compte de service doivent être sécurisés/testés avant tout déploiement accessible.

• Comptes de démo et base H2 en mémoire : non destinés à des données bancaires réelles.

Actions de remise personnelles

Créer un dépôt GitHub privé sous le nom indiqué dans l'énoncé ; remplir le Google Sheet ; réaliser des commits pendant l'épreuve à intervalles réguliers d'environ 15 minutes ; compléter ce rapport avec des captures réelles ; publier le dépôt seulement à la fin conformément aux consignes ; déposer dépôt et PDF sur Classroom. Aucun de ces actes externes n'a été exécuté automatiquement.

Références

Mohamed Youssfi : Support Architectures Micro Services avec Spring Cloud ; dépôts github.com/mohamedYoussfi/micro-services-app et github.com/mohamedYoussfi/kafka-streams-spring-cloud-stream-bdcc-enset. Documentation officielle Spring Cloud, Spring AI MCP 1.0, Keycloak, Apache Kafka ; références et URLs détaillées dans le README.